

## ASSIGNMENT-06

**Name:** Chitti Udayasathwika

**HT.No:** 2303A51322

**Batch:** 19

### Task 1: AI-Based Code Completion for Conditional Eligibility Check

The task is to generate Python code that checks voting eligibility based on a person's age and citizenship status using conditional statements.

**Prompt:** # Generate Python code to check voting eligibility based on age and citizenship.

**Code:**

```
Assign6.py > ...
1  #Task 1: AI-Based Code Completion for Conditional Eligibility Check
2  def check_voting_eligibility(age, citizenship):
3      if age >= 18 and citizenship.lower() == "yes":
4          return "Eligible to vote"
5      else:
6          return "Not eligible to vote"
7
8
9  if __name__ == "__main__":
10     age = int(input("Enter age: "))
11     citizenship = input("Are you a citizen? (yes/no): ")
12     print(check_voting_eligibility(age, citizenship))
13
```

**Result:**

```
Enter age: 12
Are you a citizen? (yes/no): yes
Not eligible to vote
```

```
Enter age: 19
Are you a citizen? (yes/no): yes
Eligible to vote
```

### Observation:

The AI-generated code correctly applies conditional logic using logical operators. The conditions are clearly defined and easy to understand. Minor optimization was done by handling case sensitivity using the `lower()` method. The logic is accurate and suitable for real-world eligibility checks.

### Task 2: AI-Based Code Completion for Loop-Based String Processing

The task is to generate Python code that counts vowels and consonants in a given string using loops.

**Prompt:** # Generate Python code to count vowels

and consonants in a string using a loop.

### Code:

```
14 #Task 2: AI-Based Code Completion for Loop-Based String Processing
15 def count_vowels_consonants(text):
16     vowels = "aeiouAEIOU"
17     vowel_count = 0
18     consonant_count = 0
19
20     for char in text:
21         if char.isalpha():
22             if char in vowels:
23                 vowel_count += 1
24             else:
25                 consonant_count += 1
26
27     return vowel_count, consonant_count
28
29
30 if __name__ == "__main__":
31     text = input("Enter a string: ")
32     v, c = count_vowels_consonants(text)
33     print("Vowels:", v)
34     print("Consonants:", c)
35
```

### Result:

```
Enter age: 19
Are you a citizen? (yes/no): yes
Eligible to vote
Enter a string: Vaishnavi Bairagoni
Vowels: 9
Consonants: 9
```

### Observation:

The AI-generated loop efficiently iterates through each character and uses conditional checks to classify vowels and consonants. The use of `isalpha()` avoids counting spaces and symbols. The logic is correct, readable, and optimized for accuracy.

### Task 3: AI-Assisted Code Completion Reflection Task

The task is to generate a complete Python program using classes, loops, and conditionals for a library management system.

**Prompt:** # Generate a Python program for a library management system using classes, loops, and conditional statements.

**Code:**

```
Assign6.py > ...
36 #Task 3: AI-Assisted Code Completion Reflection Task
37 class Library:
38     def __init__(self):
39         self.books = []
40
41     def add_book(self, book):
42         self.books.append(book)
43         print(book, "added to library")
44
45     def display_books(self):
46         if not self.books:
47             print("No books available")
48         else:
49             print("Available books:")
50             for book in self.books:
51                 print("-", book)
52
53
54 if __name__ == "__main__":
55     lib = Library()
56
57     while True:
58         print("\n1. Add Book")
59         print("2. Display Books")
60         print("3. Exit")
61
62         choice = int(input("Enter choice: "))
63
64         if choice == 1:
65             book = input("Enter book name: ")
66             lib.add_book(book)
```

```
Assign6.py > ...
7 class Library:
8
9     def display_books(self):
10         if not self.books:
11             print("No books available")
12         else:
13             print("Available books:")
14             for book in self.books:
15                 print("-", book)
16
17
18 if __name__ == "__main__":
19     lib = Library()
20
21     while True:
22         print("\n1. Add Book")
23         print("2. Display Books")
24         print("3. Exit")
25
26         choice = int(input("Enter choice: "))
27
28         if choice == 1:
29             book = input("Enter book name: ")
30             lib.add_book(book)
31         elif choice == 2:
32             lib.display_books()
33         elif choice == 3:
34             print("Exiting system")
35             break
36         else:
37             print("Invalid choice")
```

## Result:

```
Enter age: 19
Are you a citizen? (yes/no): yes
Eligible to vote
Enter a string: Vaishnavi Bairagoni
Vowels: 9
Consonants: 9

1. Add Book
2. Display Books
3. Exit
Enter choice: 1
Enter book name: Beyond the Veil of Tears
Beyond the Veil of Tears added to library
```

```
2. Display Books
3. Exit
Enter choice: 2
Available books:
- Beyond the Veil of Tears
- think like a monk
- rich dad poor dad
```

```
1. Add Book
2. Display Books
3. Exit
Enter choice: 3
Exiting system
```

## Observation:

The program allows adding books, displaying all books, and exiting the system through a menu-driven interface.

The AI-generated program effectively combines classes, loops, and conditionals. The logic is simple and functional. The menu-driven loop ensures continuous interaction. The structure is readable, and the program reflects a basic real-world library system.

## Task 4: AI-Based Code Completion for Class-Based Attendance System

The task is to generate a Python class that marks and displays student attendance using loops.

**Prompt:** #Generate a Python class to mark and display student attendance using loops.

## Code:

```
Assign6.py > ...
75 #Task 4: AI-Based Code Completion for Class-Based Attendance System
76 class Attendance:
77     def __init__(self):
78         self.records = {}
79
80     def mark_attendance(self, name, status):
81         self.records[name] = status
82
83     def display_attendance(self):
84         for name, status in self.records.items():
85             print(name, ":", status)
86
87
88 if __name__ == "__main__":
89     att = Attendance()
90
91     n = int(input("Enter number of students: "))
92     for _ in range(n):
93         name = input("Student name: ")
94         status = input("Present/Absent: ")
95         att.mark_attendance(name, status)
96
97     print("\nAttendance Record:")
98     att.display_attendance()
99
```

## Result:

```
Enter number of students: 2
Student name: Vaishnavi Bairagoni
Present/Absent: Present
Student name: Varun Sandesh Uppu
Present/Absent: Absent

Attendance Record:
Vaishnavi Bairagoni : Present
Varun Sandesh Uppu : Absent
```

## Observation

The program correctly stores and displays attendance for all students entered by the user. The AI-generated class uses a dictionary for efficient storage. Loop-based input ensures scalability. The design is simple, logical, and easy to extend for real attendance systems.

## Task 5: AI-Based Code Completion for Conditional Menu Navigation

The task is to generate a Python program using loops and conditionals to simulate an ATM menu

**Prompt: #Generate a Python program using loops and conditionals to simulate an ATM menu.**

## Code:

```
Assign6.py > ...
100 #Task 5: AI-Based Code Completion for Conditional Menu Navigation
101 balance = 10000
102
103 while True:
104     print("\nATM Menu")
105     print("1. Check Balance")
106     print("2. Withdraw")
107     print("3. Deposit")
108     print("4. Exit")
109
110     choice = int(input("Enter choice: "))
111
112     if choice == 1:
113         print("Balance:", balance)
114
115     elif choice == 2:
116         amount = int(input("Enter amount to withdraw: "))
117         if amount <= balance:
118             balance -= amount
119             print("Withdrawn:", amount)
120         else:
121             print("Insufficient balance")
122
123     elif choice == 3:
124         amount = int(input("Enter amount to deposit: "))
125         balance += amount
126         print("Deposited:", amount)
127
```

```
Assign6.py > ...
119         print("Withdrawn:", amount)
120     else:
121         print("Insufficient balance")
122
123     elif choice == 3:
124         amount = int(input("Enter amount to deposit: "))
125         balance += amount
126         print("Deposited:", amount)
127
128     elif choice == 4:
129         print("Thank you for using ATM")
130         break
131
132     else:
133         print("Invalid option")
134
135
```

## Result:

```
ATM Menu
1. Check Balance
2. Withdraw
3. Deposit
4. Exit
Enter choice: 1
Balance: 10000

ATM Menu
1. Check Balance
2. Withdraw
3. Deposit
4. Exit
```

```
Enter choice: 2
Enter amount to withdraw: 10000
Withdrawn: 10000

ATM Menu
1. Check Balance
2. Withdraw
3. Deposit
4. Exit
Enter choice: 3
Enter amount to deposit: 23800
Deposited: 23800

3. Deposit
4. Exit
Enter choice: 3
Enter amount to deposit: 23800
Deposited: 23800

ATM Menu
1. Check Balance
2. Withdraw
3. Deposit
4. Exit
Enter choice: 4
Thank you for using ATM
```

## Observation:

The ATM menu allows checking balance, withdrawing, depositing, and exiting correctly. The AI-generated logic correctly uses loops for repeated menu display and conditionals for option handling. The program is efficient and user-friendly. Minor improvements such as input validation could further enhance robustness.

